# SELENIUM AUTOMATION ASSIGNMENT REPORT

| | |
|---|---|
| Project Title | Snapdeal Automation Framework |
| Candidate Name | LOKESHWAR S R |
| Automation Type | Web Automation |
| Tool Used | Selenium with Java |
| Test Framework | TestNG |
| Framework Type | Page Object Model (POM) |
| Application Under Test | https://www.snapdeal.com/ |
| Project Repository | https://github.com/Lokeshwar-Varshan/snapdeal-automation-framework |

# TABLE OF CONTENTS


| S.no | Title | Page.no |
|---|---|---|
| 1 | Introduction | **3** |
| 2 | Objective | **3** |
| 3 | Application Under Test | **3** |
| 4 | Scope of Automation | **4** |
| 5 | Technology Stack | **4** |
| 6 | Framework Design and Architecture | **5** |
| 7 | Key Features Implemented | **6** |
| 8 | Test Scenarios Covered | **6** |
| 9 | Challenges Faced and Solutions | **7** |
| 10 | GitHub and Version Control | **7** |
| 11 | Outputs & Screenshots | **7** |
| 12 | Test Execution and Reporting | **10** |
| 13 | AI Integration | **11** |
| 14 | Conclusion | **11** |

| | | |
|---|---|---|
| 15 | References | **11** |


## 1. INTRODUCTION

This project focuses on the development of a scalable and maintainable Selenium automation framework for the Snapdeal web application. The framework is built using Java, Selenium WebDriver, and TestNG, following the Page Object Model design pattern.

The primary goal is to simulate real user behavior, automate core functionalities, and ensure application stability through structured automation practices.

---

## 2. OBJECTIVE

The objectives of this assignment are to understand the application flow, build a robust automation framework, implement best practices, and automate key user scenarios.

Additionally, the project aims to demonstrate clean code design, reusability, and integration of essential automation features such as logging, reporting, and failure handling.

---

## 3. APPLICATION UNDER TEST

The application under test is **Snapdeal**, an e-commerce platform.

URL: https://www.snapdeal.com/

The platform allows users to search for products, view product details, and perform purchase-related actions such as adding items to the cart.

---

## 4. SCOPE OF AUTOMATION

The scope of automation includes the following functionalities:

- Searching for products
- Validating search results
- Navigating to product details page
- Adding products to the cart
- Verifying cart functionality

The focus is on automating critical user journeys rather than full regression coverage.

---

## 5. TECHNOLOGY STACK

| **Component** | **Technology Used** |
|---|---|
| Programming Language | Java |
| Automation Tool | Selenium WebDriver |
| Test Framework | TestNG |
| Build Tool | Maven |
| Logging | Log4j |
| Reporting | Extent Reports |
| Driver Management | WebDriverManager |

---

## 6. FRAMEWORK DESIGN AND ARCHITECTURE

### Folder Structure

The project follows a modular structure to ensure clarity and maintainability.

automation-framework
base – Contains driver setup and base test classes
pages – Contains page object classes
tests – Contains test classes
utils – Contains utility classes
listeners – Contains TestNG listeners
resources – Contains configuration files
reports – Contains generated reports
screenshots – Contains failure screenshots

### Page Object Model Implementation

The framework follows the Page Object Model approach where each page of the application is represented by a separate class.

All element locators are encapsulated within page classes, and user actions are defined as methods. This improves maintainability and reduces code duplication.

### Base Classes and Utilities

The **BaseTest** class handles test setup and teardown, including driver initialization and configuration loading.

The **DriverFactory** class manages browser setup and provides flexibility for future scalability.

Utility classes include **WaitUtils** for synchronization**, ScreenshotUtils** for capturing screenshots, and **AIUtils** for generating dynamic test data.

## 7. KEY FEATURES IMPLEMENTED

Explicit waits are implemented using **WebDriverWait** to ensure synchronization with dynamic elements. This avoids the use of Thread.sleep and improves test stability.

Logging is implemented using **Log4j** to provide clear and structured execution logs.

Screenshot capture on failure is implemented using a **TestNG listener**, which automatically captures screenshots when a test fails.

Configuration management is handled using an external properties file that stores values such as base URL, browser type, and timeout.

Reusability is achieved through centralized utilities and modular design.

---

## 8. TEST SCENARIOS COVERED

The following test scenarios are automated:

1. **Search for a product using dynamic input**
2. **Validate that search results are displayed**
3. **Open the first product from the results**
4. **Switch to the product details page**
5. **Add the product to the cart**
6. **Click on View Cart**
7. **Validate that the cart is displayed**
8. **Verify that the cart contains at least one item**

## 9. CHALLENGES FACED AND SOLUTIONS

- One of the major challenges was handling the dynamic cart overlay. The View Cart button was not consistently clickable due to UI rendering delays. This was resolved using explicit waits and the Actions class.
- Another challenge was handling synchronization issues caused by dynamically loaded elements. This was addressed by implementing centralized wait utilities.
- Handling modal dialogs was also a challenge, which was resolved by waiting for modal visibility before performing validations.

---

## 10. GITHUB AND VERSION CONTROL

The project is hosted on GitHub.

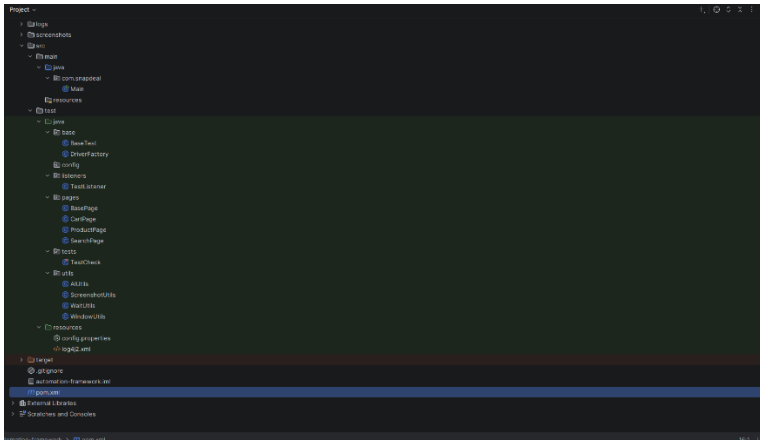
**Repository Link:** https://github.com/Lokeshwar-Varshan/snapdeal-automation-framework

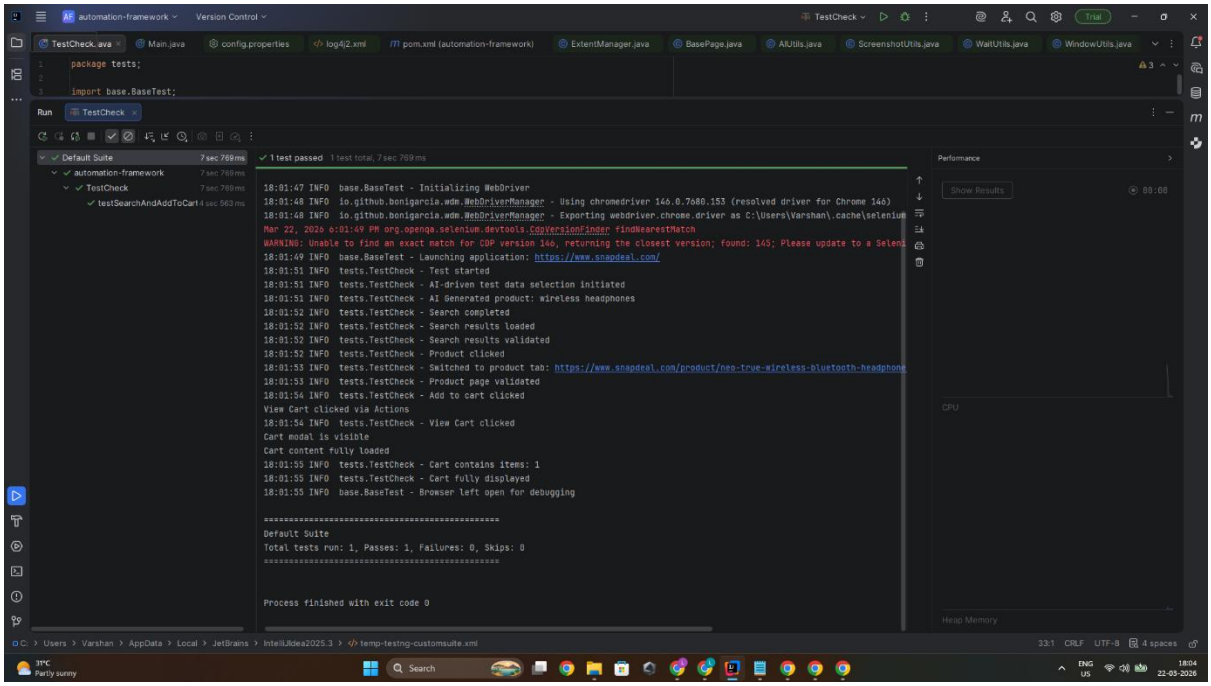
The repository follows a clean and structured approach with modular code organization.

Commits are logically structured to reflect different stages of development, such as framework setup, page implementation, and feature additions.

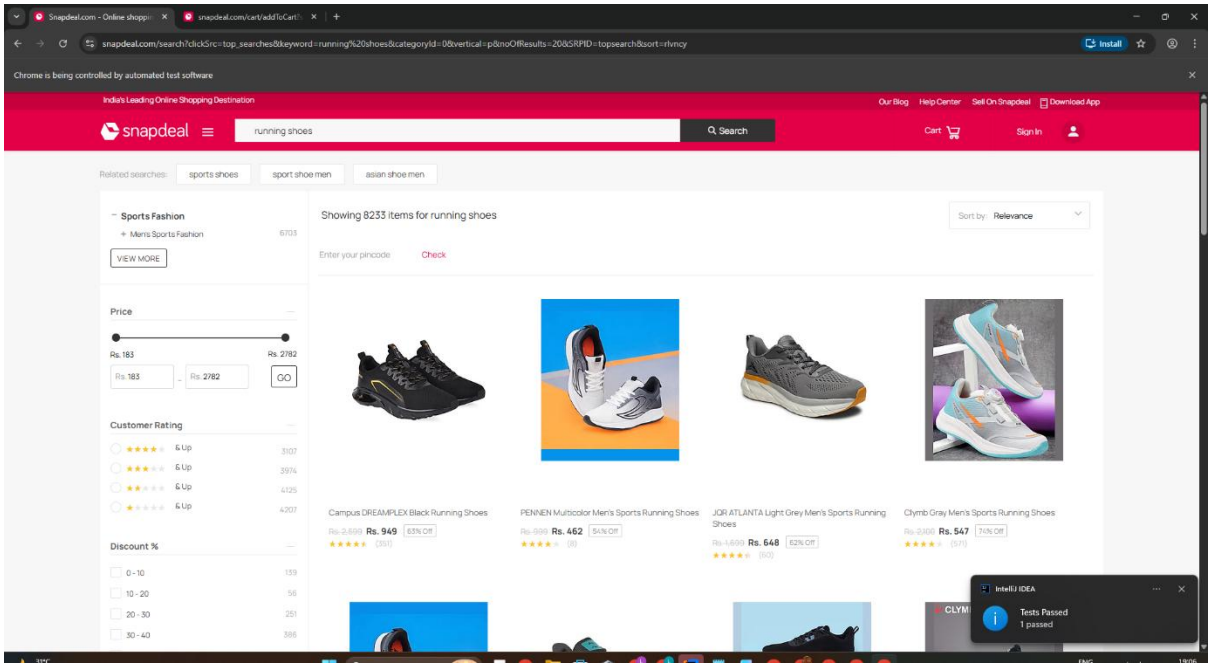
---

## 11. OUTPUT

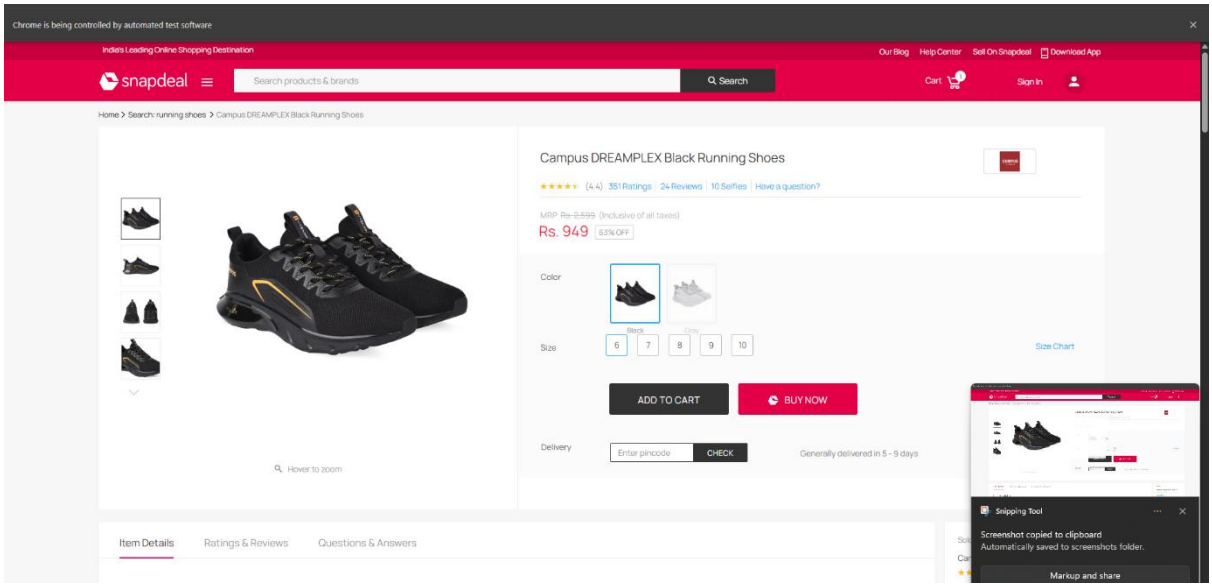




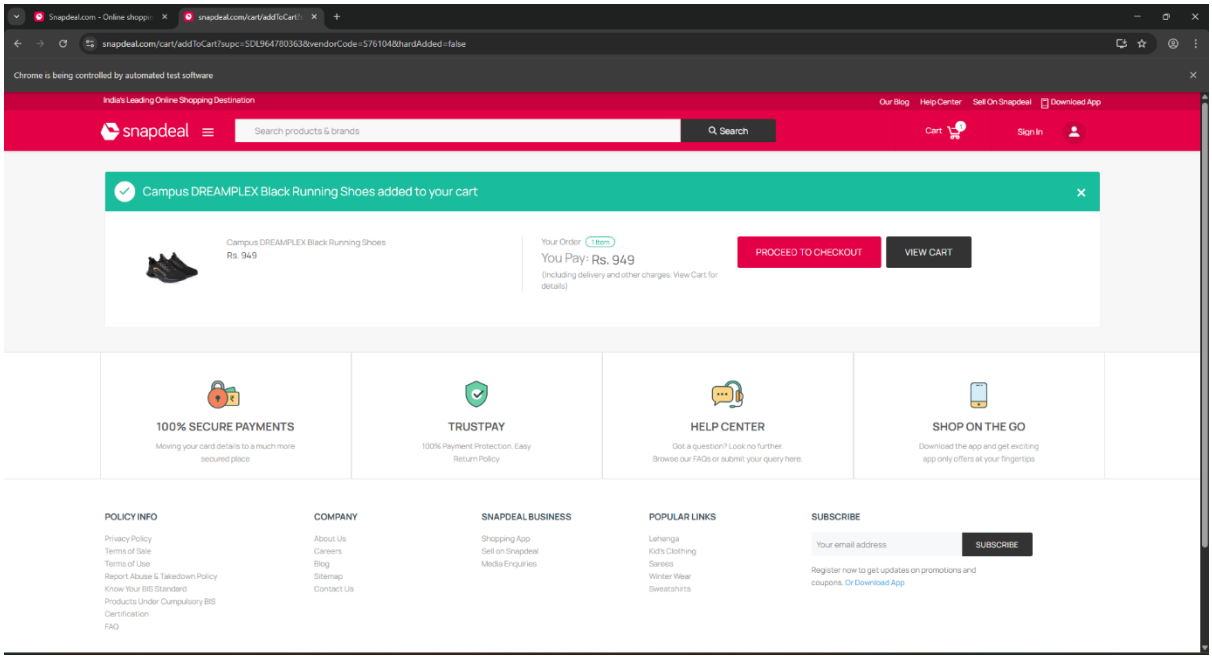

*FigNo.1 Execution screenshot (showing a successful terminal/console run)*



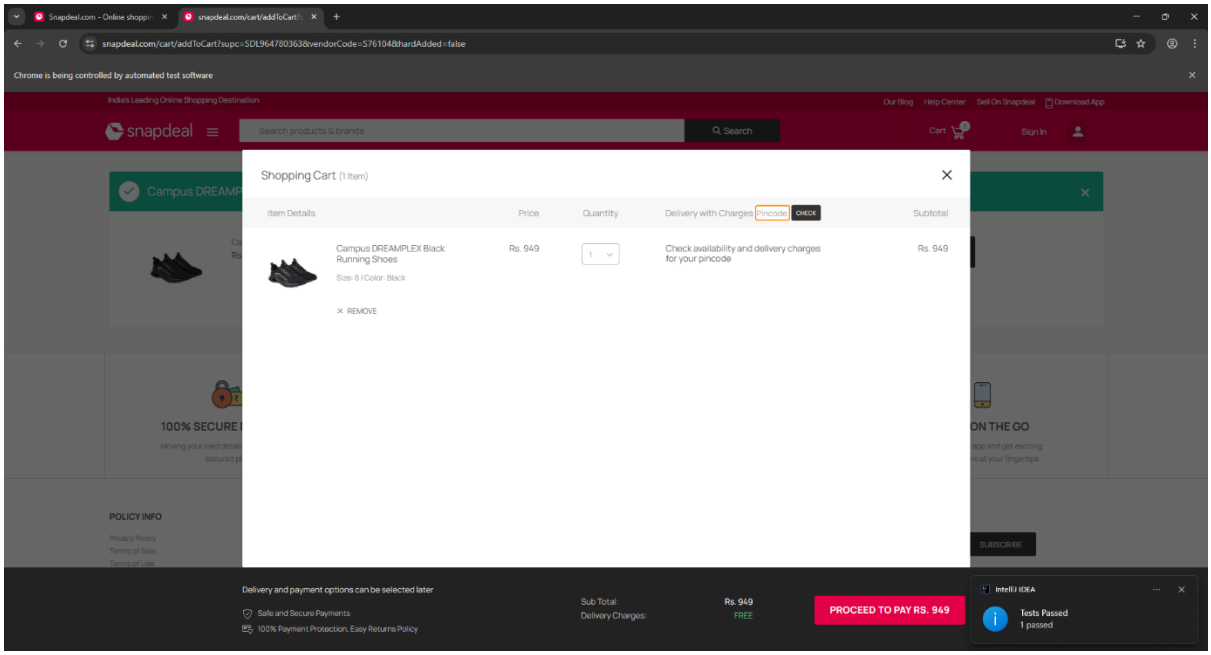

*FigNo.2 Browser screenshot (showing a snapdeal homepage with the product searched and given results)*




*FigNo.3 Browser screenshot (showing a snapdeal website -after clicking the product from the seachresult*




*FigNo.4 Browser screenshot (showing a snapdeal website -after clicking the add to cart button -shows that the product is successfully added to the cart*



***FigNo.5 Browser screenshot (showing a snapdeal website -after clicking the view cart button -shows the product details and paypayment option***

---

## 11. TEST EXECUTION AND REPORTING

Tests can be executed using Maven or directly through the IDE.

Command to run tests:

mvn clean test

Execution output is displayed in the console, showing step-by-step logs and test results.

An HTML report is generated using **Extent Reports**, which includes test status, logs, and screenshots.

The report is available in the reports folder.

---

## 12. AI INTEGRATION

The framework includes a basic **AI simulation** using a utility class that generates **dynamic product** names.

This avoids hardcoded test data and demonstrates an approach towards intelligent test automation.

---

## 13. CONCLUSION

- The project successfully demonstrates the implementation of a robust Selenium automation framework using industry best practices.

- It highlights the ability to handle dynamic elements, design scalable frameworks, and implement reliable automation solutions.
- The framework is extensible and can be enhanced further with additional features such as parallel execution and CI/CD integration.

---

## 14. REFERENCES

***Selenium Documentation:*** ***https://www.selenium.dev/***

***TestNG Documentation:*** ***https://testng.org/***

***Extent Reports Documentation:*** ***https://extentreports.com/***